

Advanced Natural Language Processing: AdvNLP

*Lesson 2: Machine Translation with
RNNs and Transformers*

Short Quiz

- Which of the following statements are true about **BPE tokenization**?
 1. It begins with a small base vocabulary and iteratively merges the most frequent adjacent symbol pairs to form larger units
 2. It guarantees that every token corresponds exactly to a linguistically complete word
 3. It can represent rare or unseen words by decomposing them into smaller learned units
 4. It requires storing every distinct word form in the training corpus as a separate vocabulary entry
 5. It always produces sequences of fixed-length tokens regardless of the input text

Short Quiz

- Which of the following statements are true about **word2vec embeddings**?
 1. It learns word representations by training a shallow neural network to predict surrounding words from a target word (or vice versa)
 2. It generates different vector representations for the same word depending on the surrounding sentence context
 3. It produces dense, low-dimensional continuous vectors where semantic similarity is reflected by geometric proximity between vectors rather than sparse high-dimensional representations
 4. With a vocabulary of 10,000 tokens and 500-dimensional embeddings, the model learns 5,000,000 embedding parameters

Table of Contents

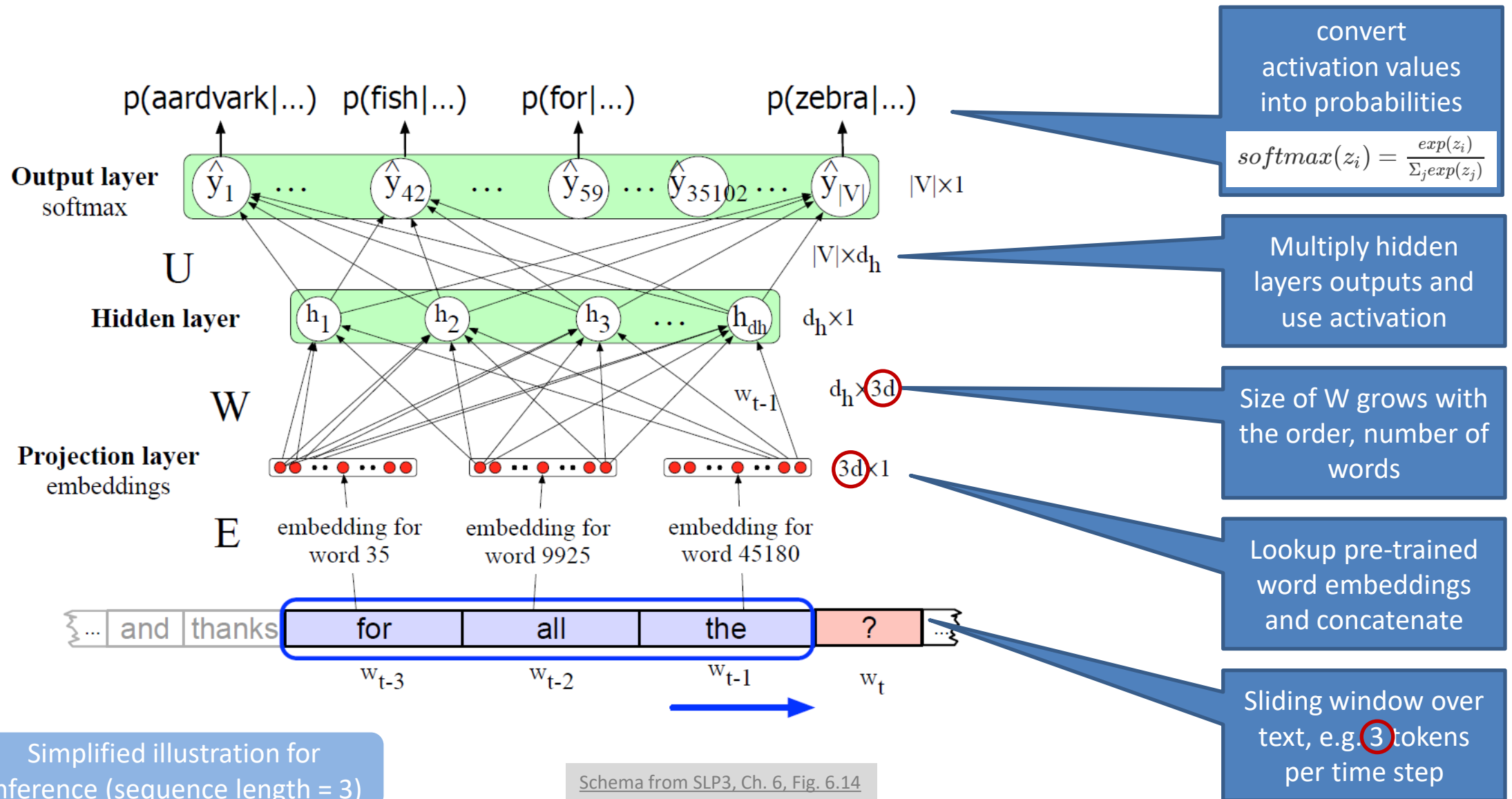
1. Recurrent neural networks for language modeling
2. RNNs with attention for machine translation
3. The Transformer for neural machine translation

1. RECURRENT NEURAL NETWORKS FOR LANGUAGE MODELING

Recap – Simple feedforward neural LMs

- Approach introduced by Y. Bengio et al. in 2003 (“A Neural Probabilistic LM”, *JMLR*, n.3)
 - “standard” feedforward neural network & Markov’s assumption
 - input: sequence of preceding words; output: probability distribution over vocabulary
- Important improvement of original idea
 - represent preceding words with their embeddings
 - better results than with words and better generalization to new data based on word similarity
- Advantages over n -gram LMs
 - no sparsity and storage problems, longer sequences/histories handled
 - higher predictive accuracy than n -gram LMs (for the same size of training data)
- Disadvantage
 - much slower to train and more difficult to interpret

Recap – FFNN LM with pre-trained embeddings



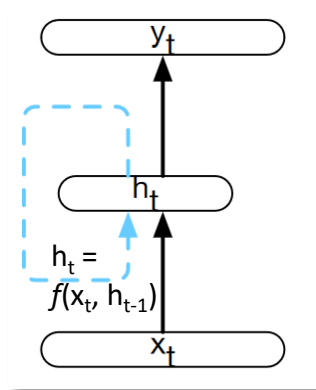
Recap – Recurrent neural networks (RNNs)

- **Strong limitation** of feed-forward networks: **fixed-size windows**

- Not long enough to model all dependencies
- size of W increases linearly with window size

- **Idea**

- present one word at a time, memorize entire past through a hidden layer, which influences *itself* at the next step
- introduced 1990, operational 2010



- **Similarities to feedforward networks**

- multiply input x_t by weight matrix, pass it through an activation function, obtain activations of hidden layer, use them to calculate output y_t

- **Key difference: “recurrent” link**

- augments input to the hidden layer with the value of the same layer **from the preceding point in time**
- **new set of weights** connect the hidden layer from the previous time step to the current one → **indicate how to use “the past” to compute the current output**
- is a **form of memory or context** that **encodes all preceding information**, i.e., allows the past to go back to the beginning of the series
- Avoids usage of fixed size window

Recap – Alternative formulations

- Dimensions of input, hidden, output layers of dimensions: d_{in} , d_h , d_{out}
- Dimensions of matrices: W is $d_h \times d_{in}$, U is $d_h \times d_h$, V is $d_{out} \times d_h$

$$h_t = g(U h_{t-1} + W x_t)$$

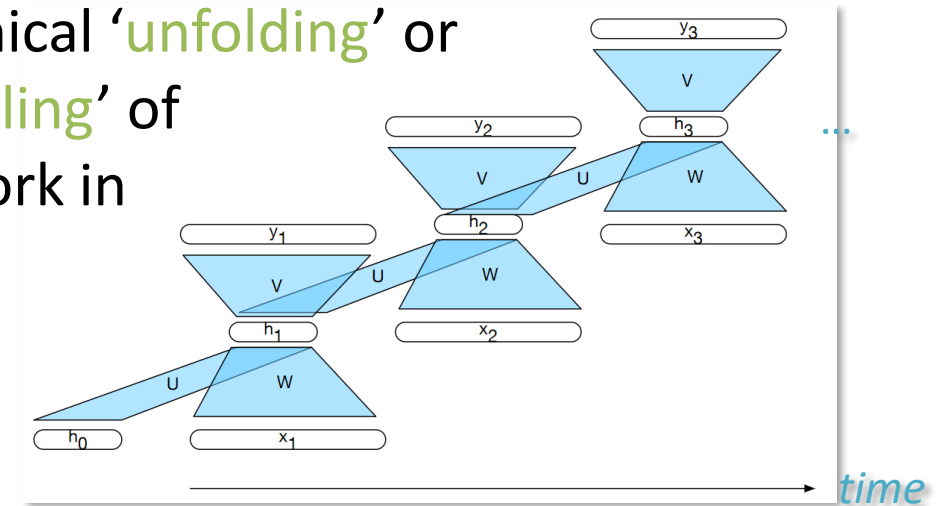
$$\hat{y}_t = f(V h_t)$$

- For classification, obtain a probability distribution over output classes as

$$\hat{y}_t = \text{softmax}(V h_t)$$

- Incremental inference algorithm: from the start of the sequence to its end

- Graphical ‘**unfolding**’ or ‘**unrolling**’ of network in time



- Training
 - recurrent connection weights are trained as all the other weights in the network
 - matrices W , U , V shared across time steps
 - method: “**backpropagation through time**” (BPTT)

Recap – RNN adaption for LMs

- Input to LM RNN are word embeddings

- add embedding weights $E \in \mathbb{R}^{d_h \times |\text{Vocab.}|}$
- rest similar to FFNN LM with $U, W \in \mathbb{R}^{d_h \times d_h}$

$$e_t = E x_t$$

$$h_t = g(U h_{t-1} + W e_t)$$

$$\hat{y}_t = \text{softmax}(V h_t)$$

- The probability that the next word is w_i from the vocabulary, is component i of $\hat{y}_t$, i.e.,

$$\hat{y}_t[i] = P(w_{t+1} = w_i | w_{1:t})$$

- The probability of a sequence of words is

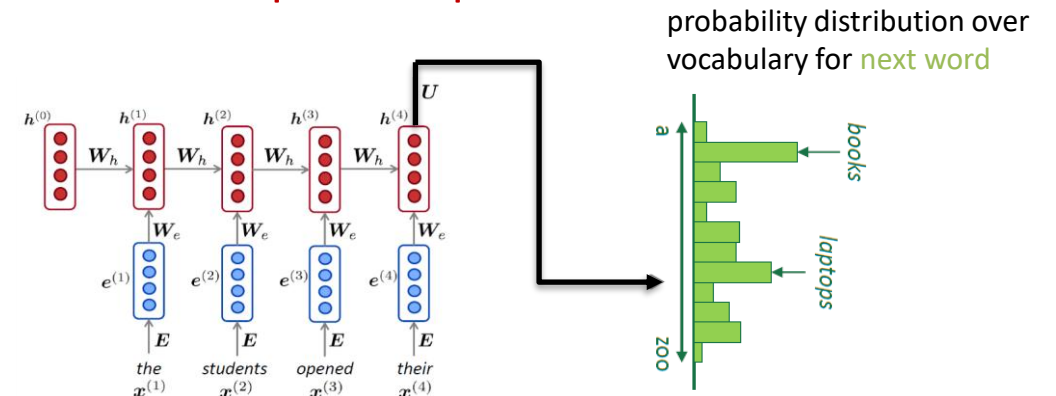
$$P(w_1, w_2, \dots, w_n) = \prod_{k=1}^n P(w_{t+1} = w_i | w_{1:t}) = \prod_{k=1}^n \hat{y}_t[i]$$

- Advantages

- constant model size for any size inputs
- same weights applied for each timestep integrating long-range time information

- Disadvantages

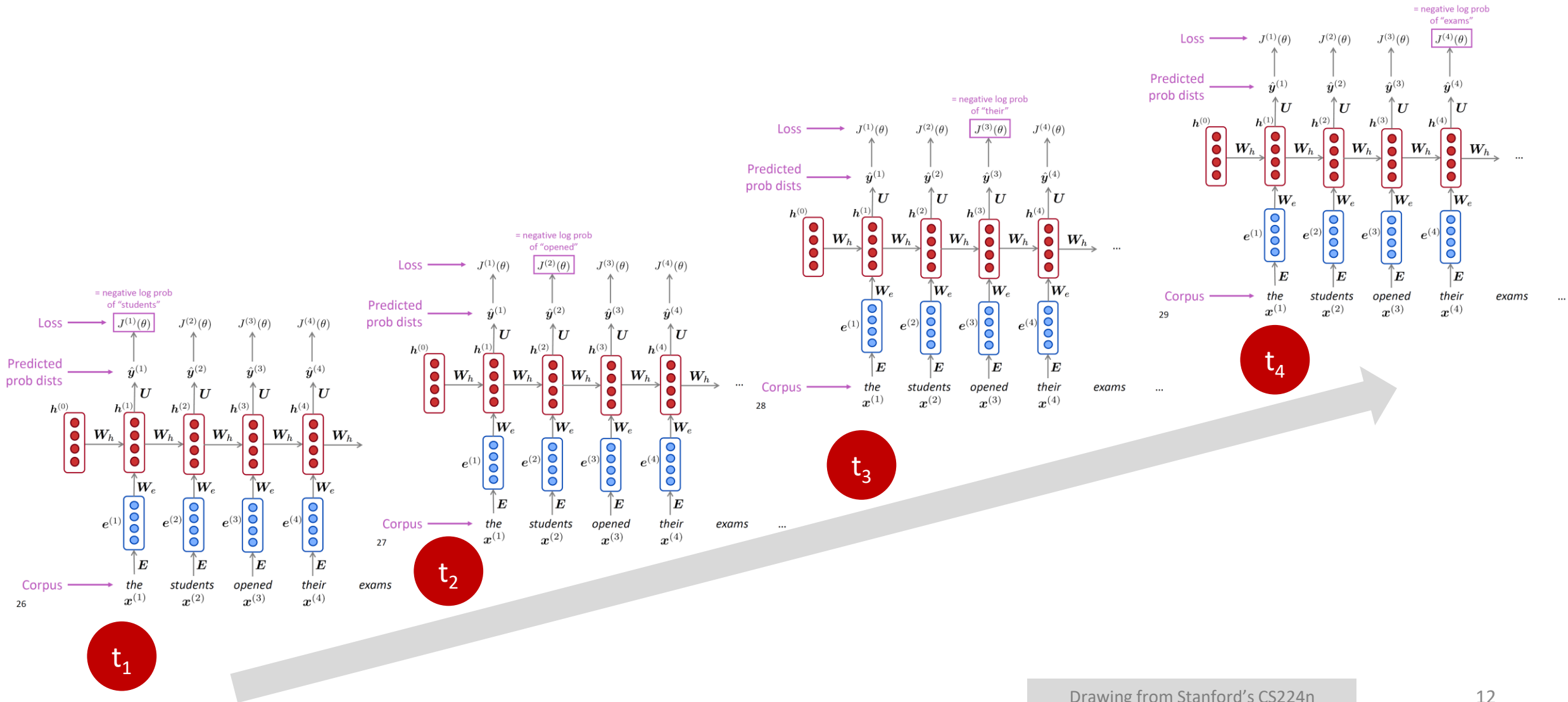
- slow training and difficulty to access to “remote past” in practice



Recap – Self-supervised training of RNN LMs (1)

- **Backpropagation in time**
 - derivative of the loss with respect to the repeated (multiplied) weight matrix W
- **Objective function: minimize CE-loss**
 - difference between predicted $\hat{y}$ and ground truth y probability distribution
$$L_{CE}(y_t, \hat{y}_t) = - \sum_{w \in V} y_t[w] \cdot \log(\hat{y}_t[w])$$
 - since the correct distribution equals 1 for the actual next word only, this simplifies to
$$L_{CE}(y_t, \hat{y}_t) = -\log(\hat{y}_t[w])$$
- **Teacher forcing**
 - use **ground truth of previous steps**, instead of the model's (imperfect) outputs
- **Stochastic Gradient Descent**
 - computing loss & gradients on entire corpus is too expensive → use batches of sentences
- **Weight tying**
 - the input embedding matrix E^T and the final layer matrix V have both “shape”, i.e., size of vocabulary times d_h
 - forcing $E = V$ **improves perplexity & reduces the number of parameters**

Recap – Self-supervised training of RNN LMs (2)

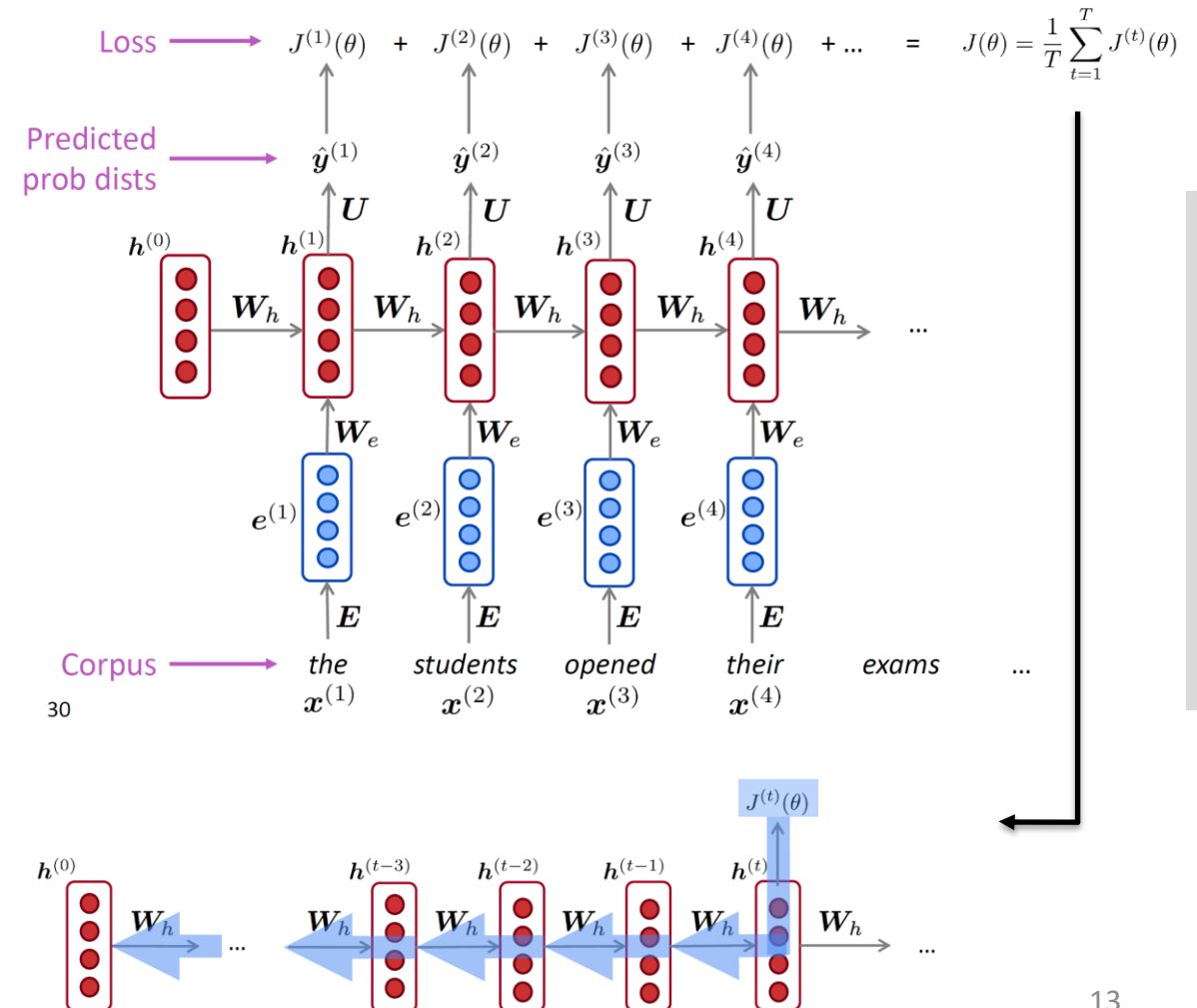


Recap – Self-supervised training of RNN LMs (3)

- BPTT is computationally expensive:
if input sequences have hundreds of timesteps, then hundreds of derivative computations are required for a single weight update.

→ This would cause learning to be slow and weights to vanish or explode

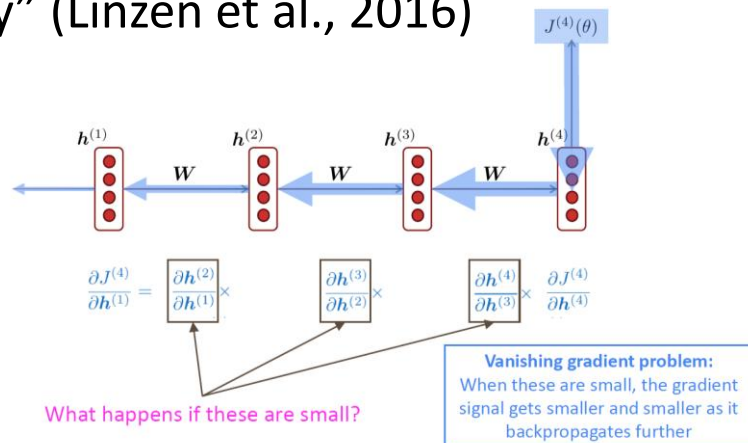
- Since training across entire corpus too expensive, sentences or documents are considered



Recap – Issues: vanishing & exploding gradients

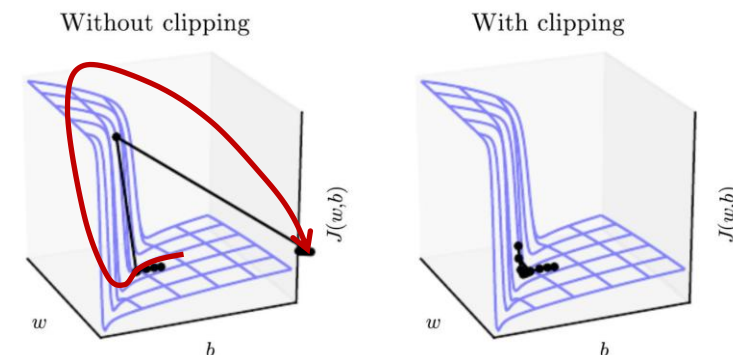
- Vanishing gradients

- gradients for remote steps in the past are much smaller than recent ones
- model weights capture well close-range, not wide-range effects
- RNN-LMs are better at learning from “*sequential* recency than from *syntactic* recency” (Linzen et al., 2016)



- Exploding gradients

- gradients (close & far) may become too big
 - then the SGD update step becomes too big, shooting far away from the optimal region
 - may even result in numeric overflows or underflows in the parameters
- Apply **gradient clipping** ... or lower learning rate

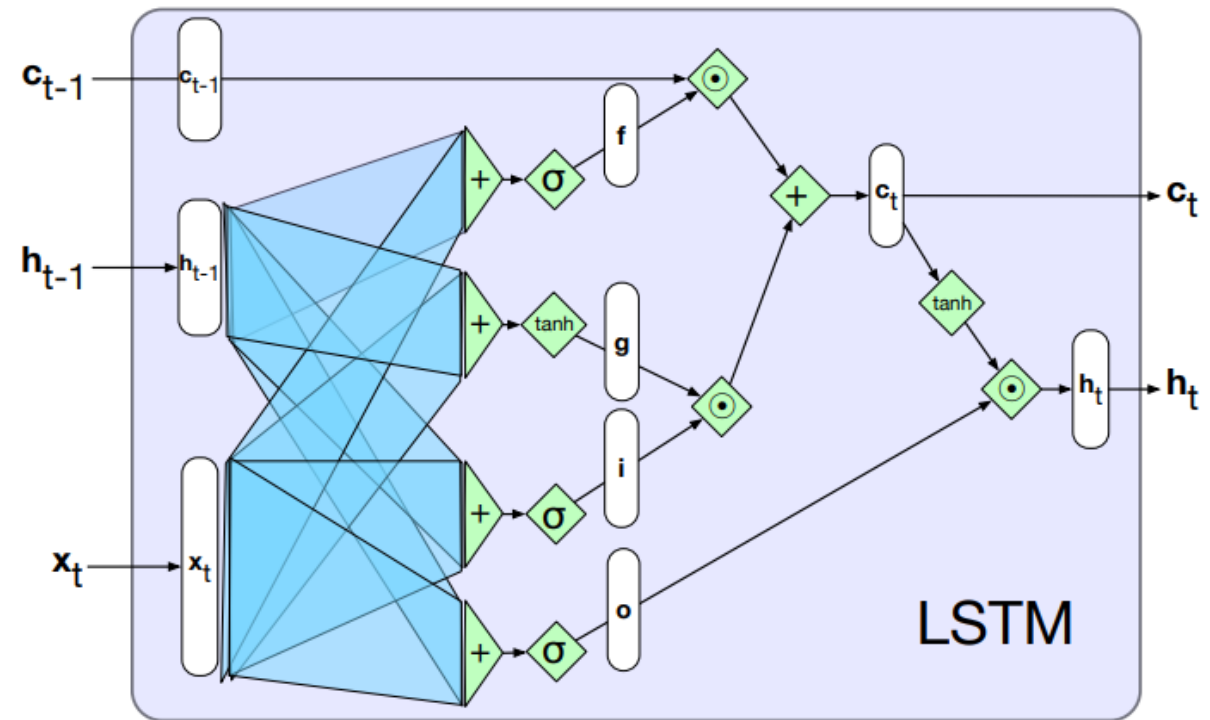


Recap – Long Short-Term Memory (LSTM) cells

- **Long-term memory**: cell state c_{t-1} helps information to flow across many time steps
- **Short-term memory**: hidden state h_{t-1} represent what is currently relevant
- **Forget gate**: f deletes information from the irrelevant context
- **Add gate**: i selects the information to add to the current context
- **Output gate**: o decides what information is required for the current hidden state

$\sigma()$: gate to control information flow $\in [0, 1]$

$\tanh()$: increase/decreased state change $[-1, 1]$



New type of neuron proposed by
Hochreiter & Schmidhuber, 1997

Drawing from SLP3, Ch. 13, Fig. 13.13

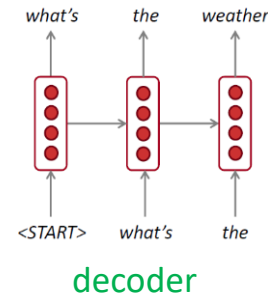
Note: LSTM's do not fully resolve the vanishing or exploding gradients issue!

2. RNNs WITH ATTENTION FOR MACHINE TRANSLATION

RNNs in sequence-to-sequence processing

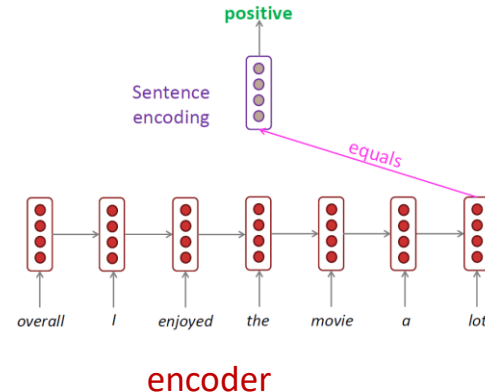
- As LMs, RNNs can **generate text auto-regressively**

- of the same genre as the training data

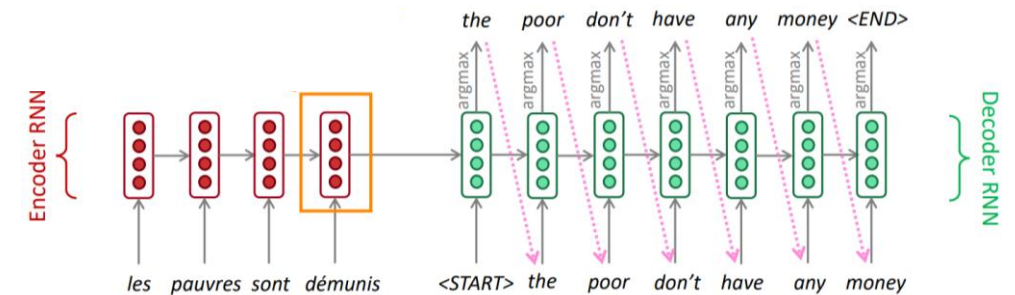


- RNNs can also **represent input text**

- using final state, or average of all states



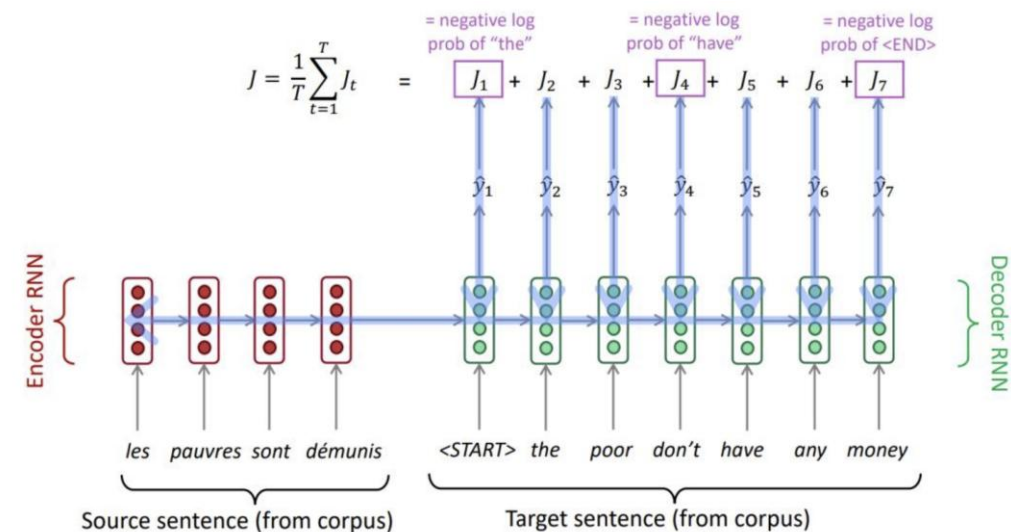
- What about combining an **encoder** (using its **final state**) with a **decoder** by conditioning it on **this state**?



Seq2seq RNN for machine translation (1)

- Machine translation (MT): given an input sentence in one language, generate an output sentence in another language
- Encoder-decoder RNN
 - a conditional language model $\Rightarrow$ the decoder outputs a probability distribution for the next word of the *output* sentence, given the *previous words* and the *encoder's* representation of the entire *input* sentence
- For MT model training
 - use a parallel corpus
 - apply teacher forcing
 - end-to-end backpropagation in time

$$P(T|S) = P(t_1|S) \cdot P(t_2|t_1, S) \cdot \dots \cdot P(t_n|t_1, \dots, t_{n-1}, S)$$



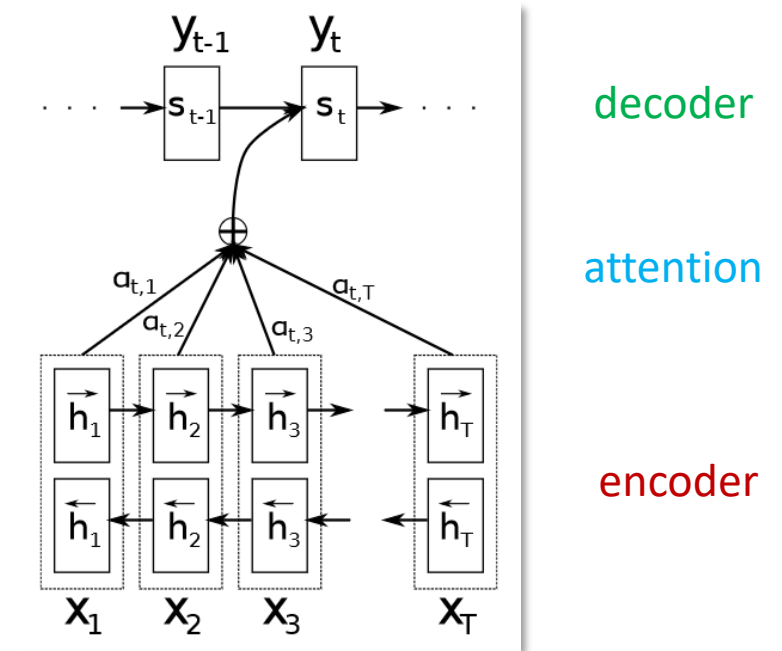
Seq2seq RNN for machine translation (2)

- First RNN to demonstrate competitive translation capabilities: Sutskever et al., “Sequence to Sequence Learning with NNs”, *Proc. of NeurIPS 2014*
 - multi-layer RNN (= stacked) with LSTM units (possibly also GRUs), 4 layers deep
 - the depth allows the network to compute more complex representations
- Seq2seq RNNs are useful for many other NLP tasks (e.g., summarization or dialogue)
 - single neural network to be optimized end-to-end (no individual subcomponents optimization)
 - no feature engineering (no linguistic domain knowledge), same method for all language pairs
- Crucial limitation of seq2seq RNN:
 - all source sentence content squeezed into encoder RNN’ final state (bottleneck between encoder & decoder) → translation quality decreases significantly with the length of the source sentence



Seq2seq RNNs with attention

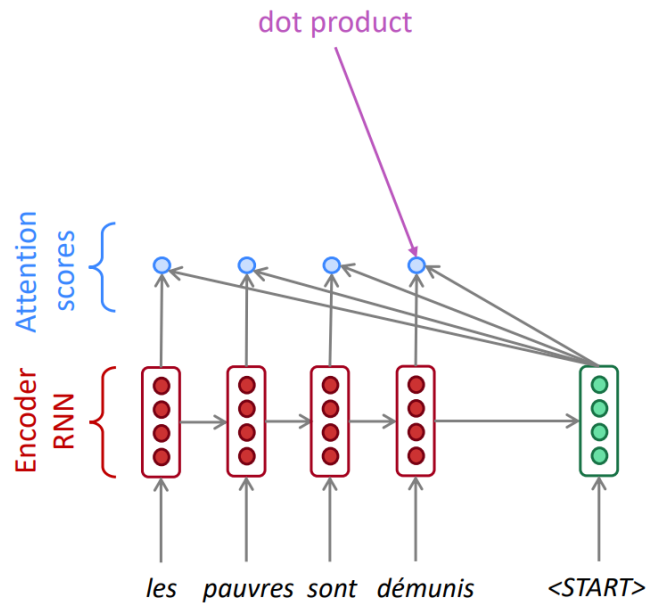
- Idea:
 - use RNN with a bidirectional encoder and a left-to-right decoder
 - on each step of the decoder, focus on a particular part of the source sequence
- Enable the decoder to access directly, with a combination of weights, each of the hidden states of the encoder
 - weights = “attention” on the respective states
 - “attention” is a vector of the size of the text span (maximum size, use padding for shorter sentences)
 - “attention” weights depend on the word to generate



Detailed view of RNN with attention for MT

Abigail See, 2018, <http://web.stanford.edu/class/cs224n/lectures/lecture10.pdf>

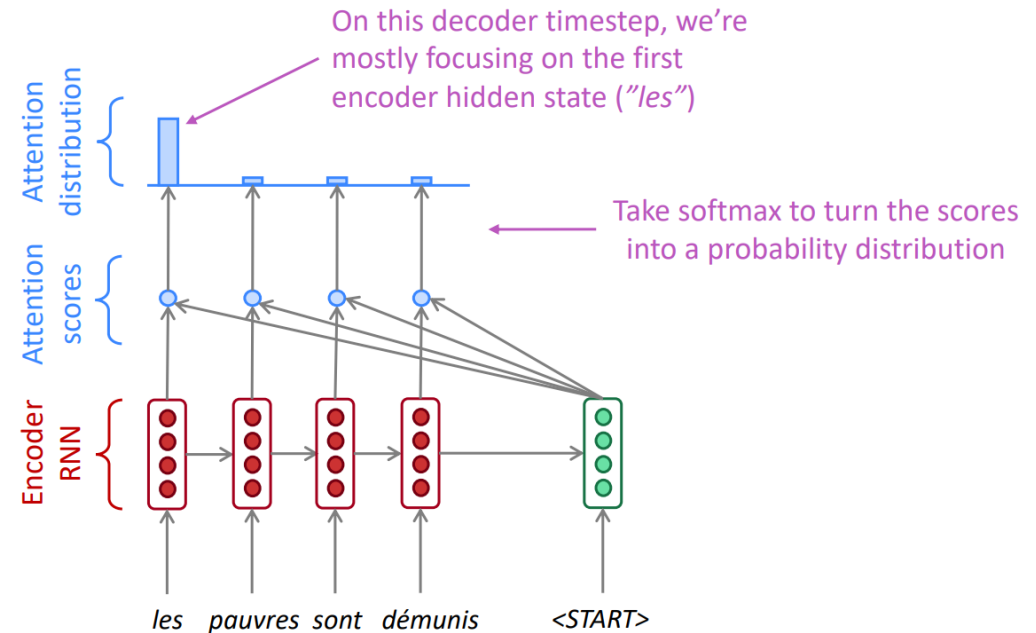
1



Time step

t_1

2



Time step

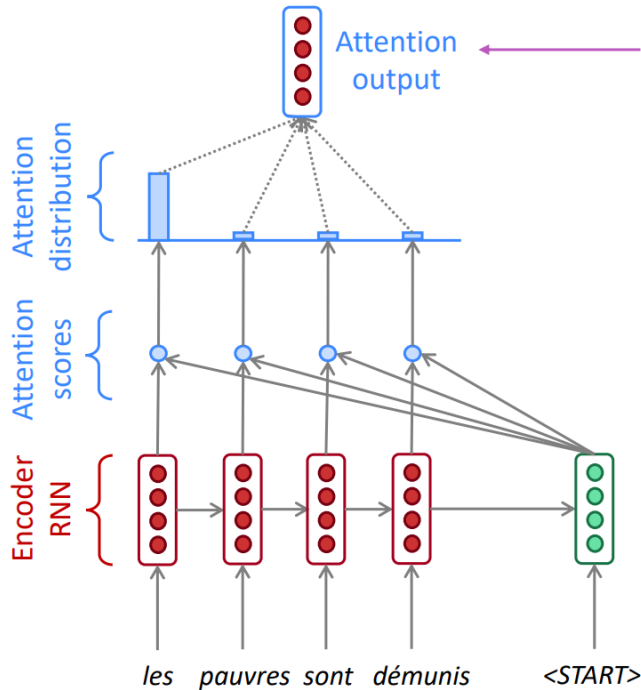
t_1

Decoder RNN

Detailed view of RNN with attention for MT

Abigail See, 2018, <http://web.stanford.edu/class/cs224n/lectures/lecture10.pdf>

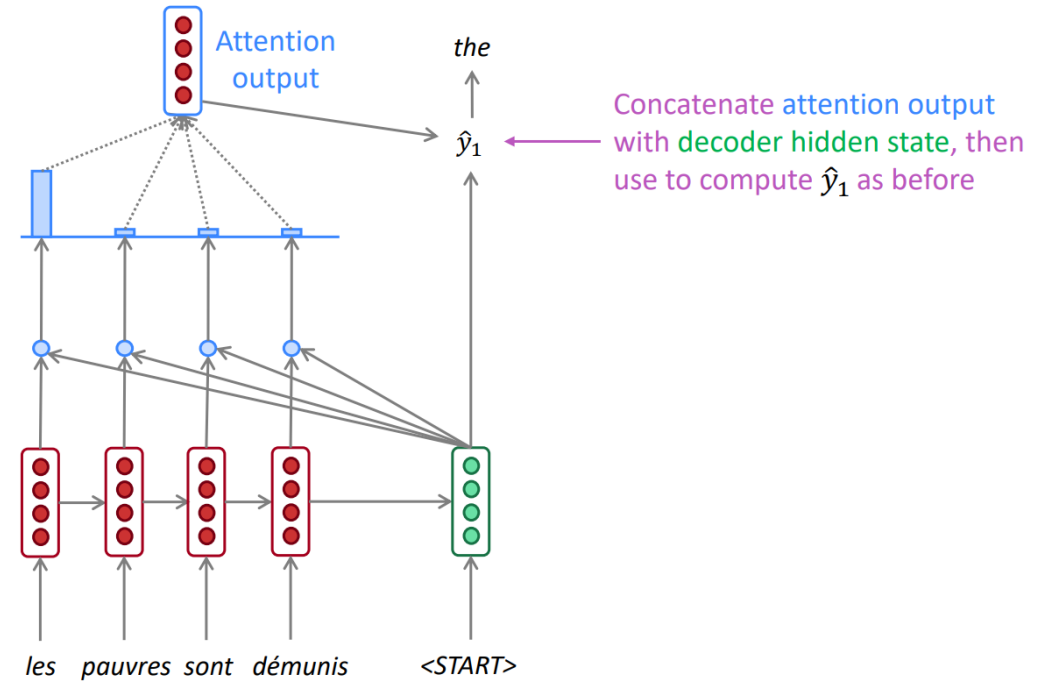
3



Time step

t_1

4



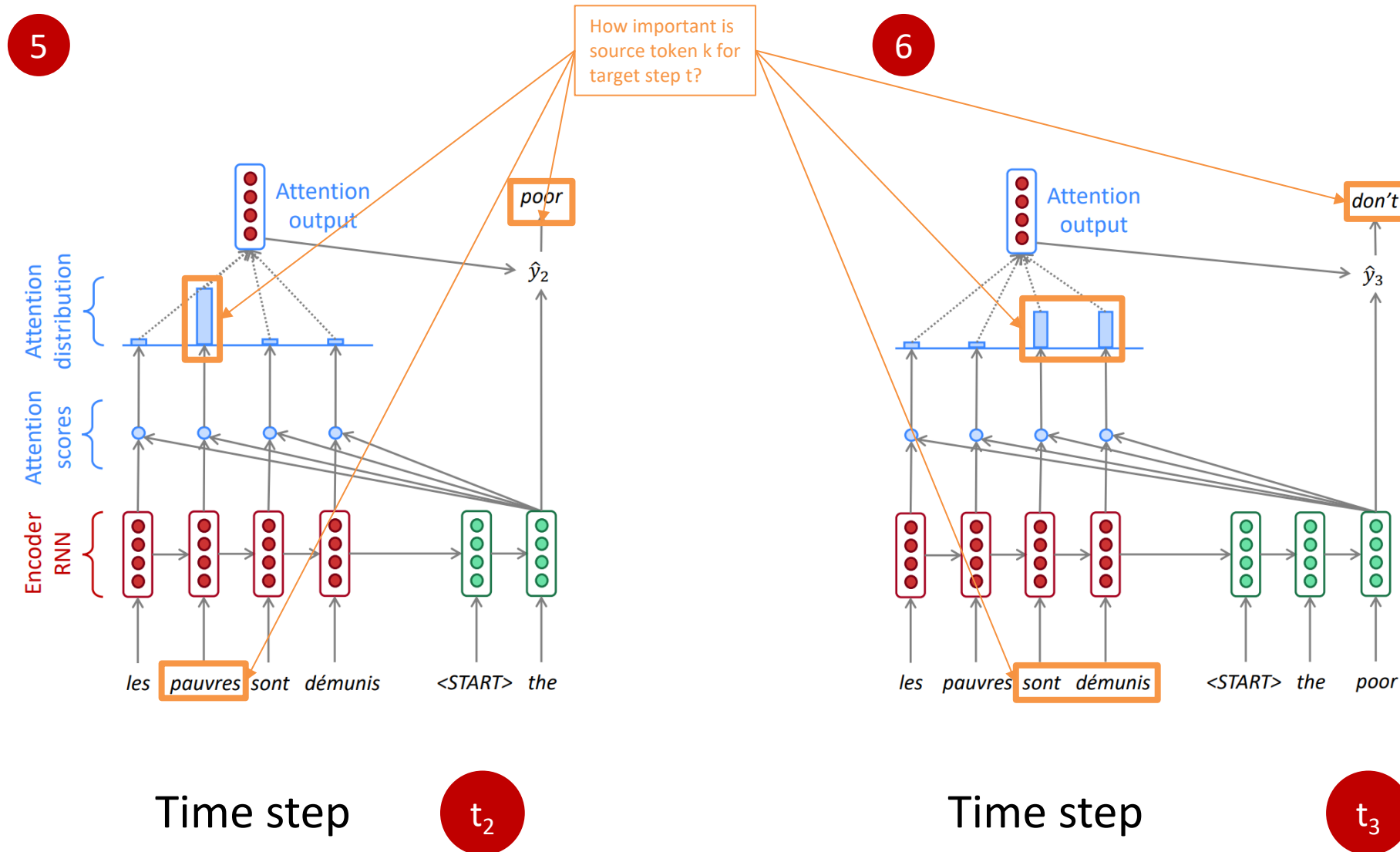
Time step

t_1

Decoder RNN

Detailed view of RNN with attention for MT

Abigail See, 2018, <http://web.stanford.edu/class/cs224n/lectures/lecture10.pdf>

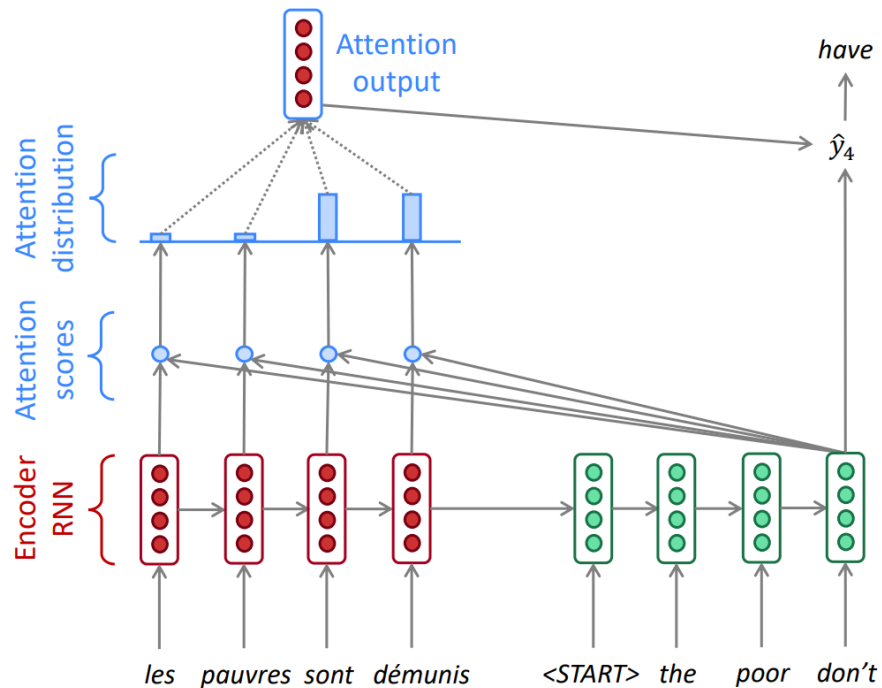


Decoder RNN

Detailed view of RNN with attention for MT

Abigail See, 2018, <http://web.stanford.edu/class/cs224n/lectures/lecture10.pdf>

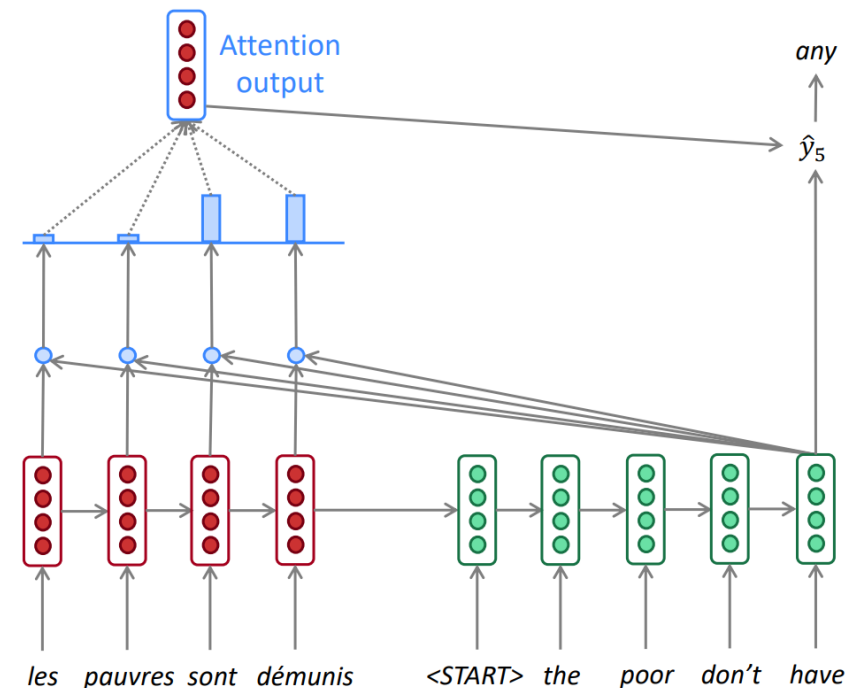
7



Time step

t_4

8



Time step

t_5

Decoder RNN

Mathematical formulation

- There are N hidden states of the **encoder**
 $h_1, \dots, h_N \in \mathbb{R}^H$
 - H is the dimension of the hidden layer,
 N is the length of the input sequence
- To generate output token t , consider state $s_t \in \mathbb{R}^H$ of the hidden layer of the **decoder**
- **Attention scores** for each **encoder** hidden state

$$e^t = [s_t^T \cdot h_1, \dots, s_t^T \cdot h_N] \in \mathbb{R}^N$$

- **Attention distribution**: probs derived from scores in e^t using component-wise softmax

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

- **Attention output** (“context vector”) at step t of the **decoder**: weighted sum of all **encoder’s** hidden states

$$a_t = \sum_{i=1}^N \alpha_i^t \cdot h_i \in \mathbb{R}^H$$

- **Context vector** is concatenated with **hidden state** to form a **new hidden state** $[a_t, s_t] \in \mathbb{R}^{2H}$ to be used instead of a regular RNNs s_t

- defined as above, attention doesn’t need more parameters, but other ways to compute attention require additional weights (d_1, d_2 are dims of the encoder/decoder)

Multiplicative attention: $e_i = s^T W h_i \in \mathbb{R}$

- Where $W \in \mathbb{R}^{d_2 \times d_1}$ is a weight matrix

Additive attention: $e_i = v^T \tanh(W_1 h_i + W_2 s) \in \mathbb{R}$

- Where $W_1 \in \mathbb{R}^{d_3 \times d_1}$, $W_2 \in \mathbb{R}^{d_3 \times d_2}$ are weight matrices and $v \in \mathbb{R}^{d_3}$ is a weight vector.
- d_3 (the attention dimensionality) is a hyperparameter

Attention – example calculation

Source: «*il m'a entarté*»



Target: «*he hit me with a pie*»

Decoder hidden state $s_1 : [5, 5, 5]$

Context vector a_1

Word token	Encoder hidden state	Attention scores	Attention distribution	Attention output components
il	[1.0, 1.0, 1.0]	15.0	0.875	[0.875, 0.875, 0.875]
m'	[0.5, 0.8, 1.0]	11.5	0.026	[0.013, 0.021, 0.026]
a	[1.0, 1.0, 0.5]	12.5	0.072	[0.072, 0.072, 0.036]
entarté	[0.8, 0.5, 1.0]	11.5	0.026	[0.021, 0.013, 0.026]

context vector for first decoder token "he" attends mostly to encoder hidden state of word "il"

[0.981, 0.981, 0.963]

1 dot product
→ scores

$$e^t = [s_t^T \cdot h_1, \dots, s_t^T \cdot h_N] \in \mathbb{R}^N$$

2 softmax
→ probab. distribution

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

3 lin. combination
→ context vector

$$\alpha_i^t \cdot h_i$$

$$a_t = \sum_{i=1}^N \alpha_i^t \cdot h_i$$

Remarks on attention

- Attention

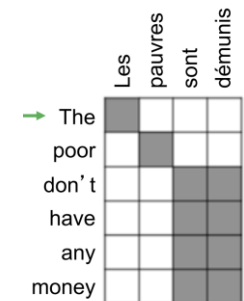
- computes a weighted sum of the values dependent on the decoder's query
- results in a fixed-size representation of a set of representations (the values), dependent on another representation (the query)
- summarizes information contained in the values, with a focus selected by the query
- the output mostly contains information from hidden states that received high attention

- RNNs with attention are limited w.r.t. parallelization 

- Attention is applicable in many modalities, architectures and tasks

- Attention

- improves NMT performance (the decoder focuses on parts of the source)
- solves the bottleneck problem and helps with the vanishing gradient problem (the decoder looks directly at the entire source and provides shortcut to far away states)
- provides some interpretability (attention distribution reveals the decoder's focus)
- offers alignment for free w/o explicit training

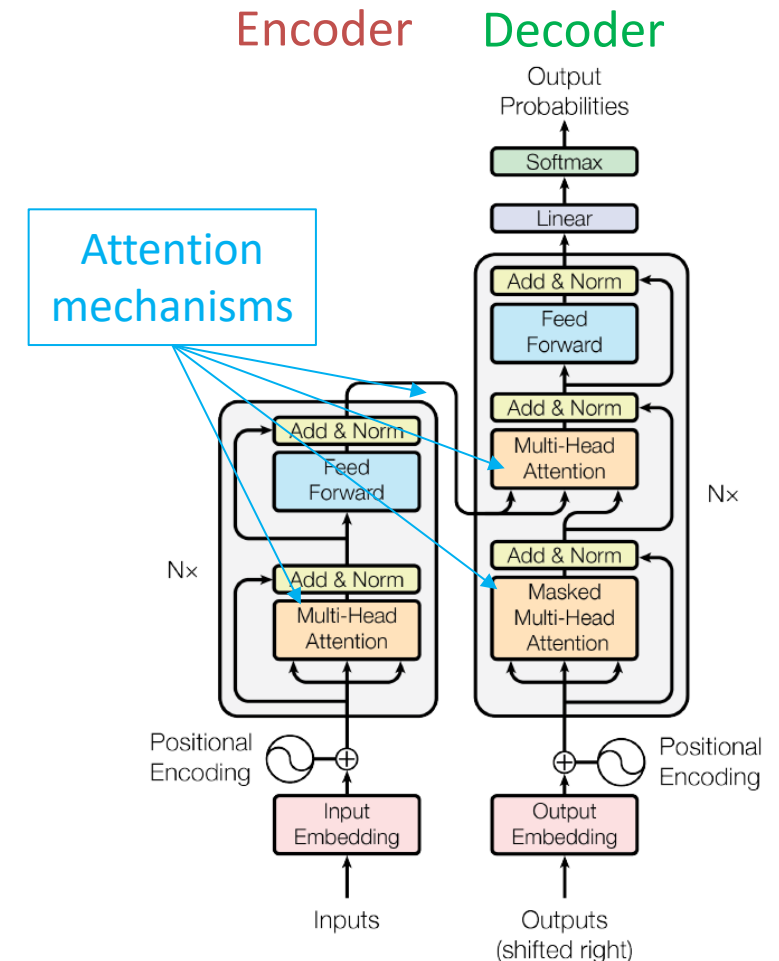


	Les	pauvres	sont	démunis
→ The	dark			
poor		dark		
don't			dark	
have				dark
any				dark
money				dark

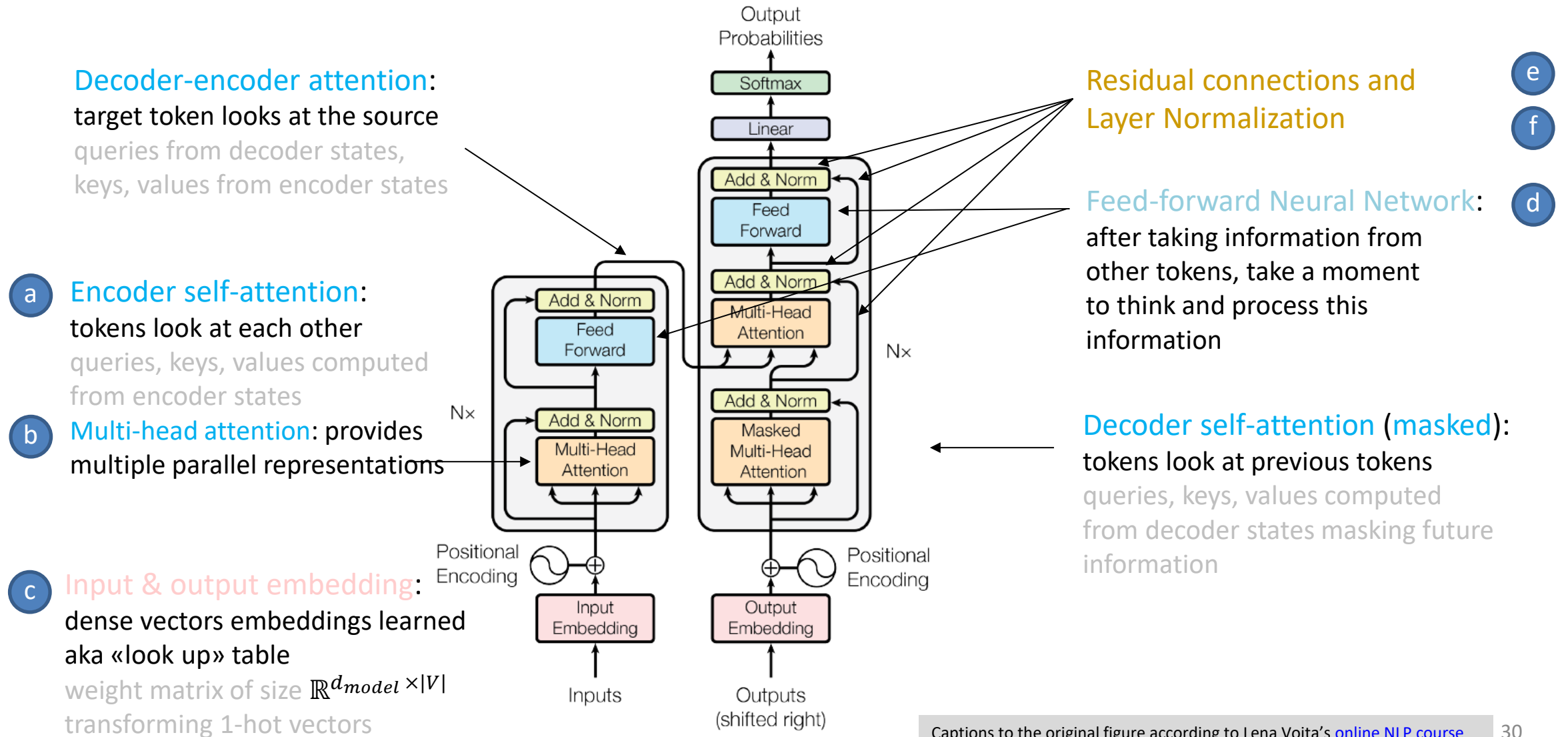
3. “ATTENTION IS ALL YOU NEED”: THE TRANSFORMER FOR NEURAL MACHINE TRANSLATION

The Transformer

- Encoder-Decoder model for MT without recurrence
- Encoder: input sentence $\rightarrow$ abstract representation**
 - N (originally $N=6$) stacked blocks of multi-head attention layers plus fully connected FF NN layers
- Decoder: output sentence up to word $w_{i-1} \rightarrow P(w_i)$**
 - N (originally 6) stacked blocks of *masked* multi-head attention layers, with inputs from encoder and *past* decoder layers + fully-connected FF NN layers
- Note: like the RNN, the model does not really generate a translation but assigns probabilities on word w_i given **input** and **output** $w_1, \dots, w_{i-1}$.
- Decoder strategy** needed to build the translation ⚠



Role of each part of the Transformer



Self-attention

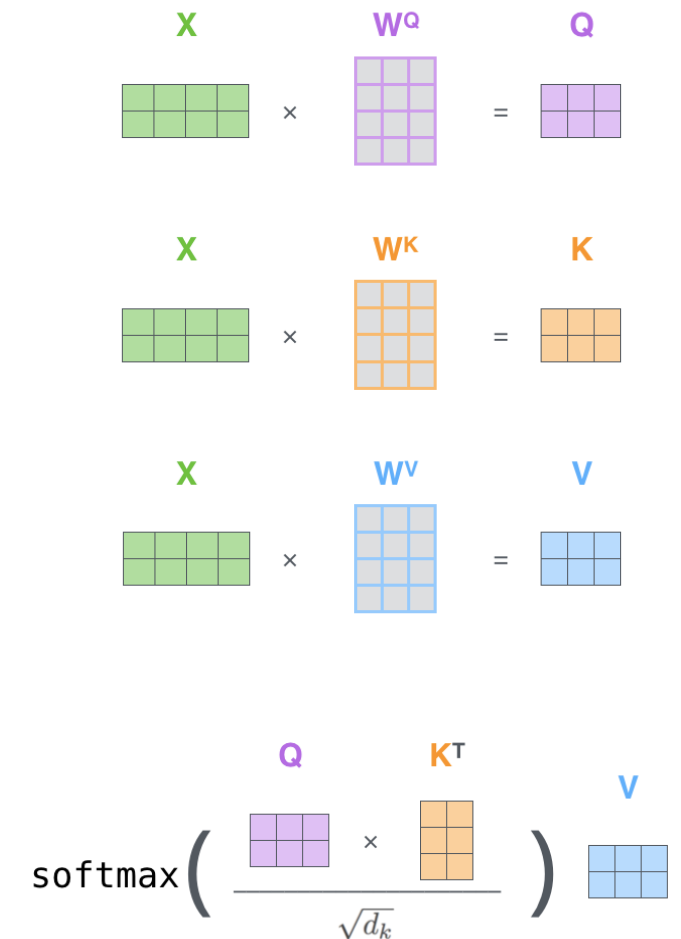
- Self-attention learns a vector representation of each word based on the representations of potentially distant words in the sentence
- The input vector of the word token in position i is noted x_i from which 3 vectors are derived
- Each of these token vectors plays **three roles**
 - Query q_i : “Hey there, do you have this information” focus
 - Key k_i : “Yes, give me a large weight” compare
 - Value v_i : “Here’s the information” output
- This logic is applied in every encoder sublayer
- All input vectors can be packed into one matrix X



Self-attention

All computations are parallelizable by taking advantage of efficient matrix multiplication

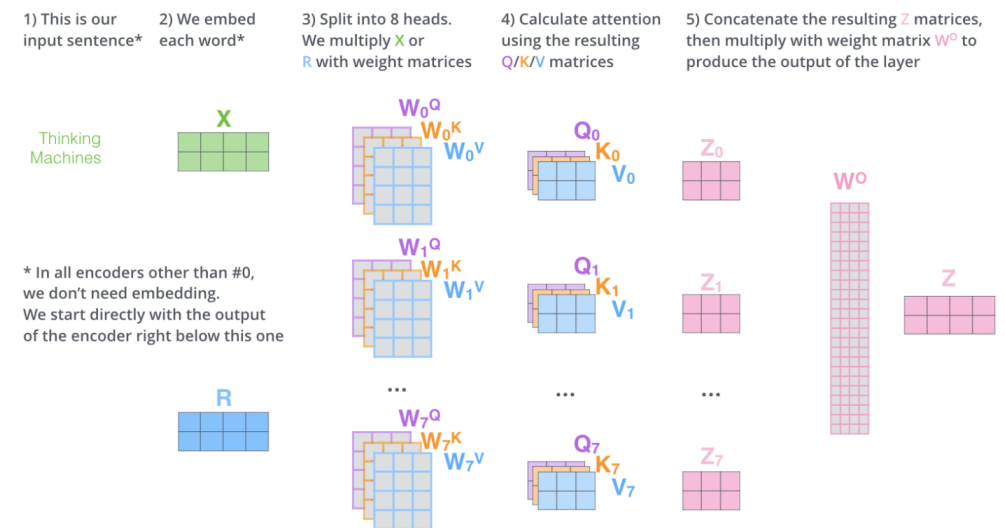
- The self-attention head has three learnable weight matrices W^Q , W^K and W^V (all $\in \mathbb{R}^{d_{model} \times d_k}$)...
- ... which are used to project each token vector x_i
 - $q_i = W^Q \cdot x_i$, $k_i = W^K \cdot x_i$, $v_i = W^V \cdot x_i$
- For a given query q_i we compute across all tokens j
 - Attention scores: $s_{ij} = q_i \cdot k_j / \sqrt{d_k}$ ($\sqrt{d_k}$ rescaling!)
 - Attention distribution: $a_{ij} = \text{softmax}(s_{ij})$ ($\sum_j a_{ij} = 1$)
 - Attention output: $z_i = \sum_k a_{ki} \cdot v_k$
- All vectors can be updated in one matrix operation
 - $Z = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \cdot V$ ($Q = W^Q \cdot X$, $K = W^K \cdot X$, $V = W^V \cdot X$)



Multi-head attention (MHA)

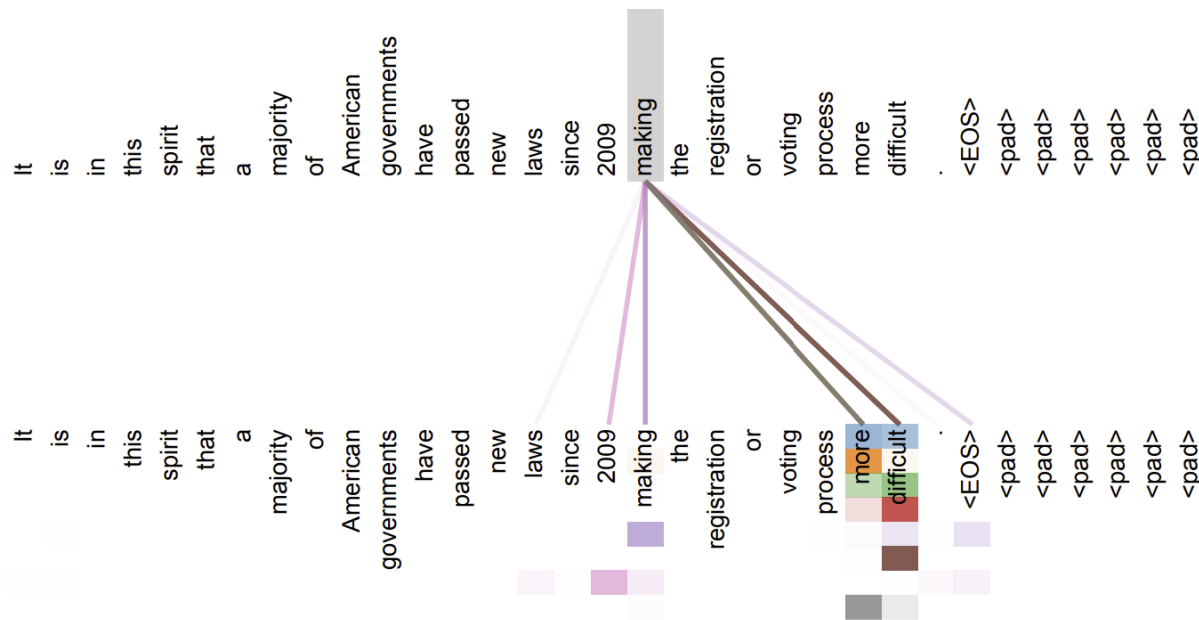
- Each word can be part of many relations... managing multiple parallel representations allows to extract different constraints for each word
- Each query, key & value is split into $h = 8$ heads, s.t. for $d_{model} = 512$, vectors q_i, k_i, v_i are all of dimension 64
- The resulting Z matrices are concatenated and projected through $W^O \in \mathbb{R}^{hd_v \times d_{model}}$
 - $Concat[Z_0, Z_1, \dots, Z_7] \cdot W^O$

- MHA can specialize and capture different linguistic relationships (e.g. syntax, semantics, reference) which are **recombined**



Multi-head attention (MHA) - Examples

Encoder self-attention in layer 5: **Long-distance dependencies** in the verb 'making', completing the phrase 'making...more difficult'



Encoder self-attention in layer 5 (heads 5 & 6): Isolated attentions enabling **anaphora resolution** for the word 'its'

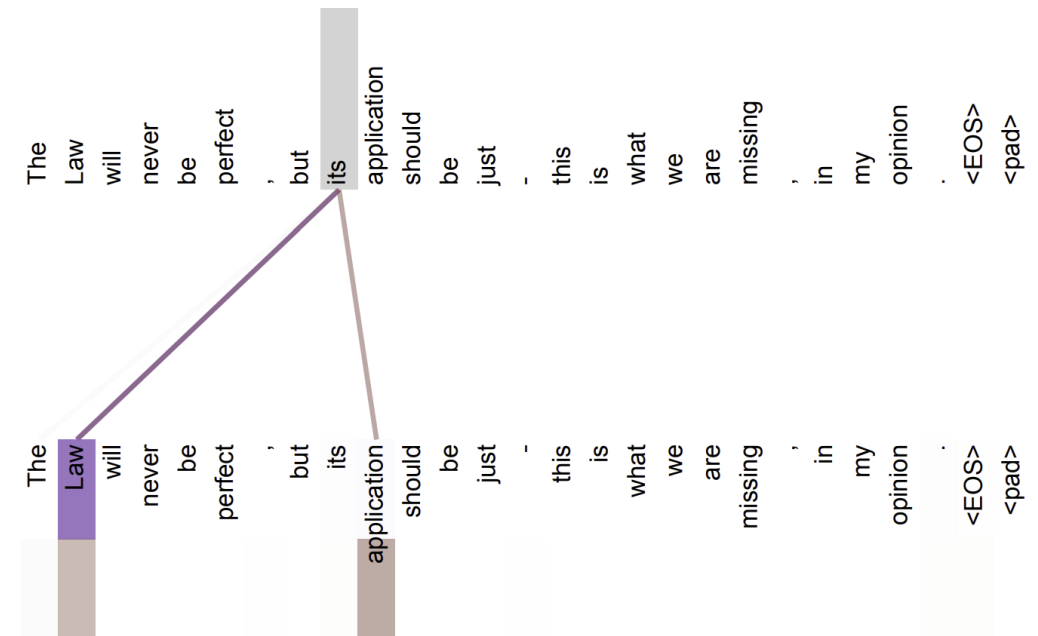
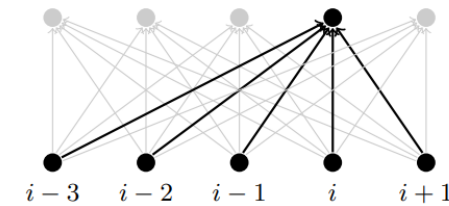


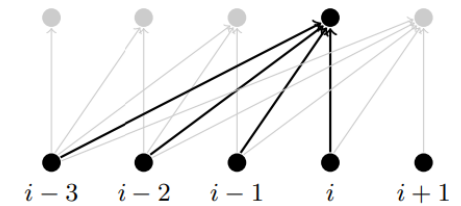
Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

Masked self attention

- Masked attention in the **decoder** prevents each token from attending to future tokens by introducing a mask.
 - Modifying attention scores of future tokens are set to a very large negative number
 - The modified attention scores are then passed through the softmax function, effectively zeroing out the attention to future tokens.



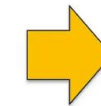
(a) Encoder-side Self-attention



(b) Decoder-side Self-attention

$$\frac{QK^T}{\sqrt{d_k}}$$

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229



$$\frac{QK^T}{\sqrt{d_k}} + M$$

	THE	CAT	IS	ON	A	CHAIR
THE	0.268	-∞	-∞	-∞	-∞	-∞
CAT	0.124	0.278	-∞	-∞	-∞	-∞
IS	0.147	0.132	0.262	-∞	-∞	-∞
ON	0.210	0.128	0.206	0.212	-∞	-∞
A	0.146	0.158	0.152	0.143	0.227	-∞
CHAIR	0.195	0.114	0.203	0.103	0.157	0.229

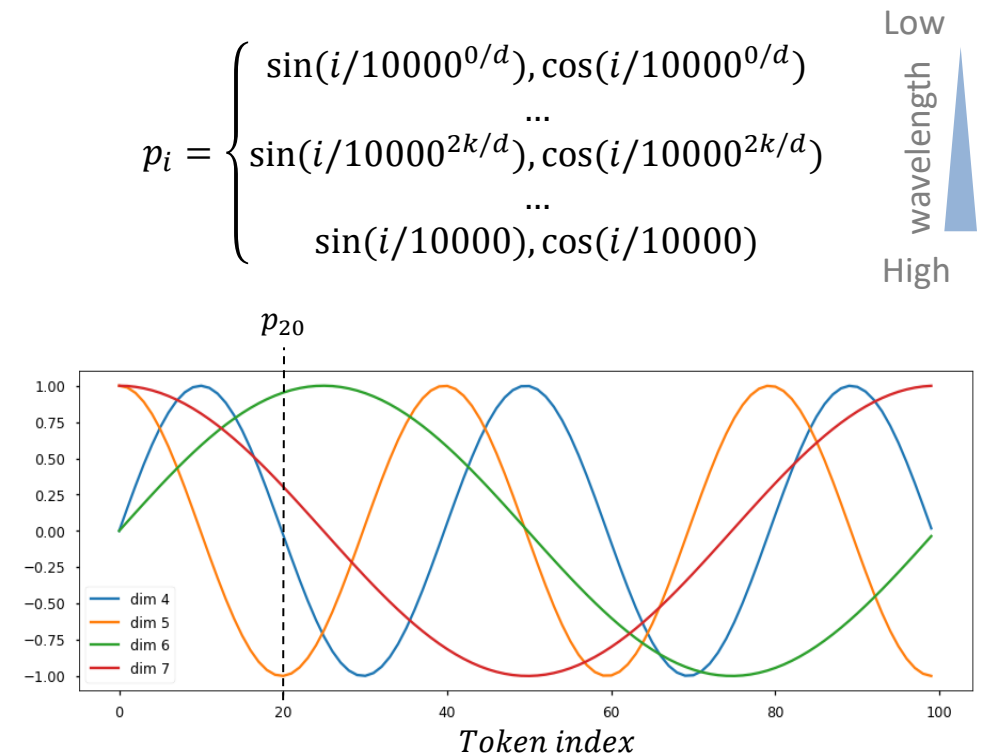
$$Z_{masked} = \left(\frac{Q \times K^T}{\sqrt{d_k}} + M \right) \cdot V \quad \text{with} \quad M_{ik} = \begin{cases} 0 & (i < k) \\ -\infty & (i \geq k) \end{cases}$$

Short Quiz

- Which of the following statements are true about **attention**?
 1. To calculate attention weights for input I_2 , you would use key k_2 , and all queries
 2. To calculate attention weights for input I_2 , you would use query q_2 , and all keys
 3. We scale the QKT product to bring attention weights in the range of $[0,1]$
 4. We scale the QKT product to allow for numerical stability

Positional encoding

- Transformer blocks don't hold information about word order
- Solution with trigonometric functions
 - p_i = encoding for token i , $d = d_{model}$
- Position is encoded as vector $\in \mathbb{R}^{d_{model}}$, s.t.
 - Token positions in a sentence are unique
 - Encoding generalizes to arbitrary sentence length and capture relative token distances
 - Encoding allow training convergence, i.e., bounded, balanced positive & negative values
 - Encoding captures token distances rather than absolute position in sequence



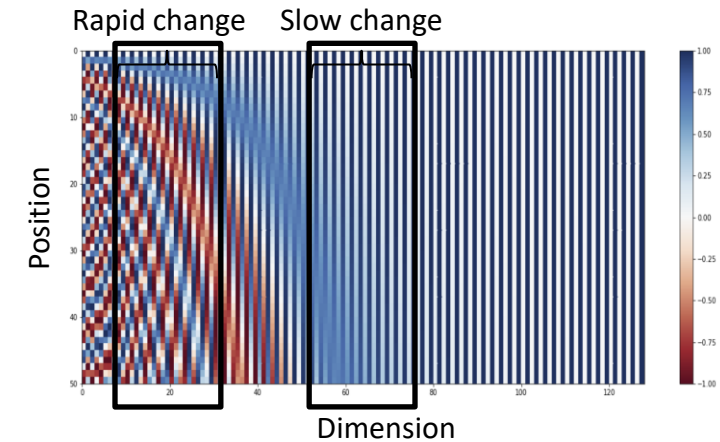
Positional encoding

- Trigonometric positional encodings at different positions are linearly related, s.t. the model learns to attend to relative positions ...based on

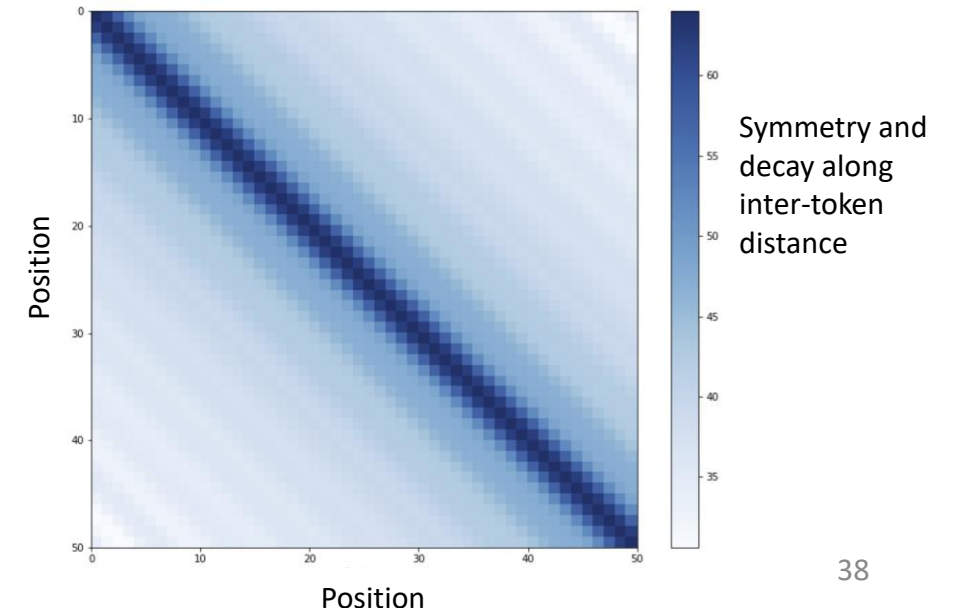
$$M \cdot \begin{pmatrix} \sin(\omega_k \cdot i) \\ \cos(\omega_k \cdot i) \end{pmatrix} = \begin{pmatrix} \sin(\omega_k \cdot (i + j)) \\ \cos(\omega_k \cdot (i + j)) \end{pmatrix}$$

- The position vector $p_i \in \mathbb{R}^{d_{model}}$ is added once to the input embedding $x_i = x_{i,input} + p_i$
 - Low dimensions encapsulate positional information, higher dimensions token-specific information

Row-wise stacking of PE vectors

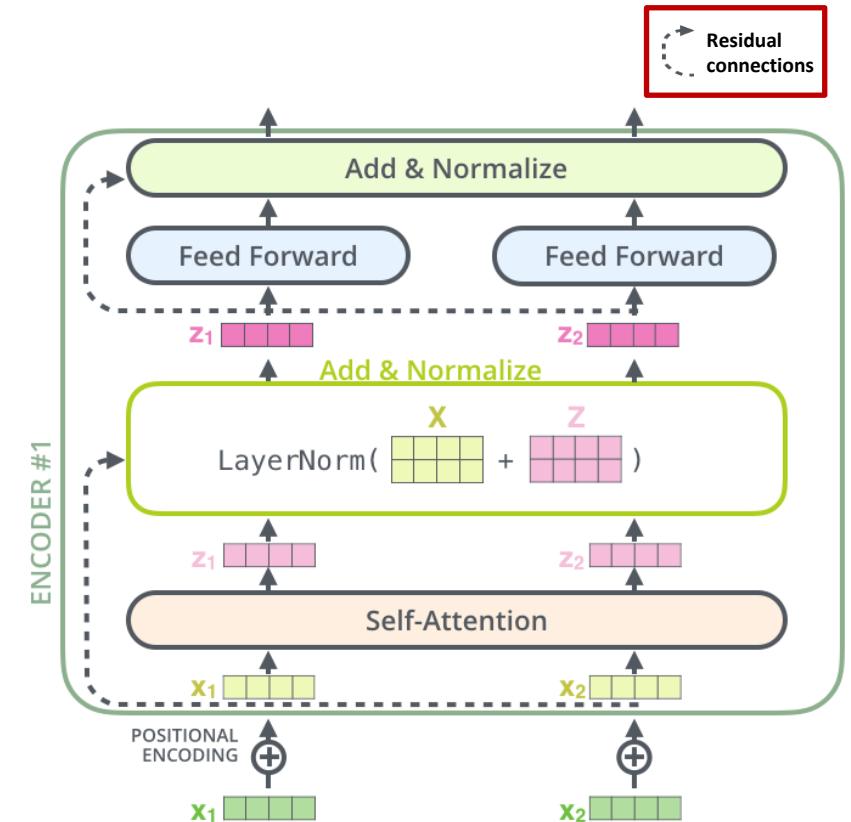


PE vector dot products



Non-linearities, information flow & robustness

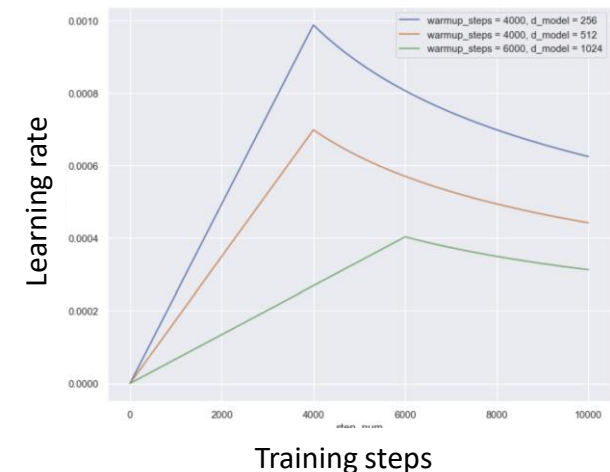
- Feed Forward NNs introduce non-linearities via
 - $FFNN(x_i) = \max(0, W_1 \cdot z_i + b_1) \cdot W_2 + b_2$
 with shared trainable matrices W_i & bias terms b_i
- Residual connections () “ease the gradient flow through a network and allow stacking layers” making the loss landscape smoother
- Layer Normalization ([Ba et al, 2016](#)) normalizes each word vector via $g \odot \frac{(x_i + z_i) - \mu}{\sigma + \epsilon} + bias$ stabilizing the dynamics & reducing training time



Transformer training (1)

- Training is **challenging** w.r.t. architecture, hyperparameters and compute
 - **Sensitive hyper-parameters** e.g., network shape, dropout, optimizer, label smoothing, ...
 - **Tokenization & Batching**: BPE for MT G → E; Word-Piece for F → E with shared source-target vocabularies of 30-40k tokens
 - **Learning rate schedule**: linear increase followed by gradual decay

$$lr = d_{model} \cdot \min(n_{step}^{-0.5}, n_{step} \cdot n_{warmup}^{-1.5})$$
 - **Batching & padding**: Sentence batched together by approximate sequence length with ca. 25k source / 25k target tokens per batch
 - **Steps/Time**: machine with 8 NVIDIA P100 GPUs, trained base models with 100,000 steps in 12 hours, big models with 300,000 steps in 3.5 days

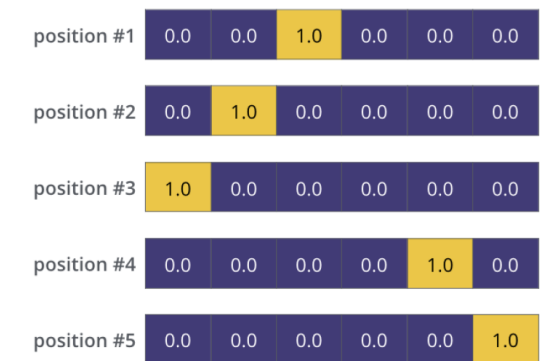


Transformer training (2)

- Proceed from the beginning of the sequence, with **teacher forcing**, and gradually unmask desired output from left to right
 - compare predicted next word with ground truth: **cross-entropy as loss function** and **backpropagate loss** (compute gradients)
 - copy the ground-truth word as the next correct word in decoder's input
 - go through the entire batch from start to end, then update the weights of the network

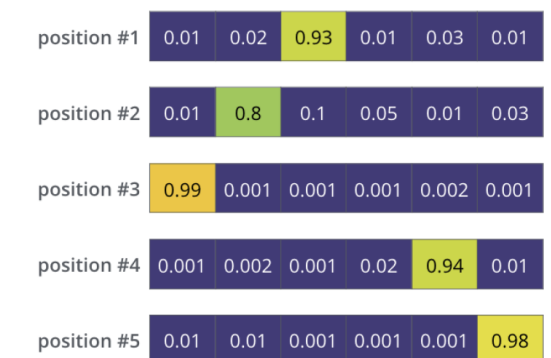
Target Model Outputs

Output Vocabulary: a am I thanks student <eos>



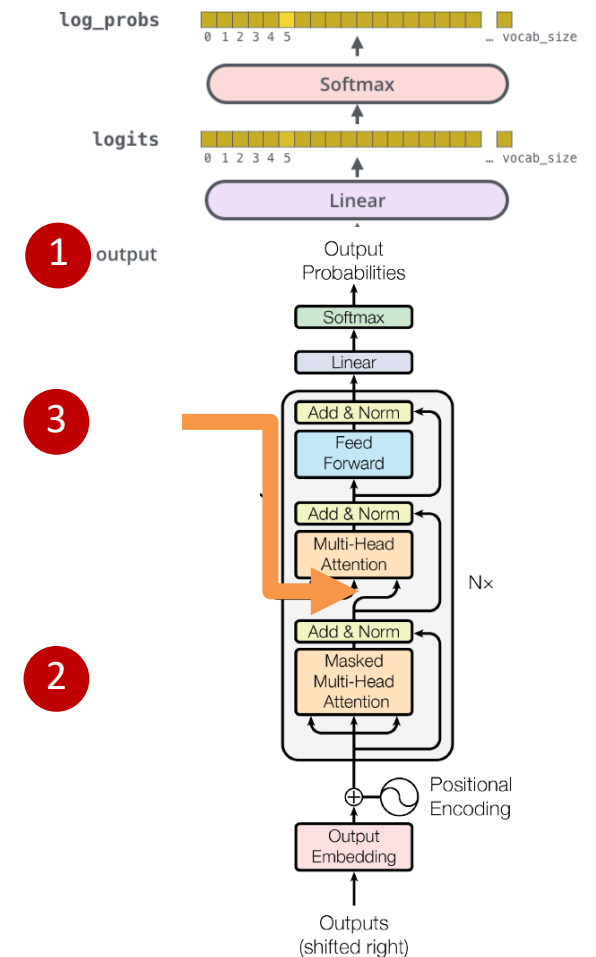
Trained Model Outputs

Output Vocabulary: a am I thanks student <eos>



Transformer inference

- 1 Thanks to the final linear and softmax layers, the **decoder outputs a probability distribution** over the vocabulary for the next word given a sequence (cf. decoding strategies)
 - Attention in the decoder reminds of **RNNs with attention**
- 2 in the translated (target) sequence, the decoder attends to all source words + only previous target words (see **masked self attention**)
- 3 Cross- attention-to-encoder head: k_i and v_i are from the **top layer of the encoder**, q_i is from the **previous layer of the decoder**



Transformers summary (1)

- Transformers allow for **parallelization during training**
 - All **encoder sublayers** process **all input tokens** in parallel and self-attention has full visibility of the entire input sentence. With **batching**, this is also parallelized **across multiple sentences**
 - **Position encodings** injected in bottom layers is responsible to capture **sequential token ordering**
 - Outputs from the last encoder layer are fed to all decoder layers using cross-attention, s.t. the decoder can attend to **all encoder tokens at once for each decoder token**
 - All **decoder layers** process the full target sequence in parallel leveraging masked-self attention and teacher forcing
- During **inference**, tokens are generated one at a time and selected through autoregressive decoding (discussed later in this course)

Transformers summary (2)

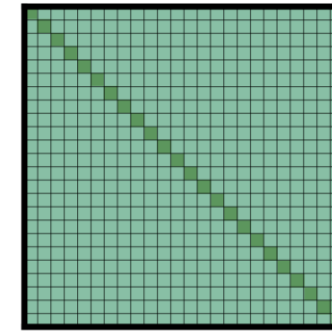
- Text inputs are tokenized and embedded in d_{model} dimensions for a vocabulary of size $|V|$ with $d_{model} \cdot |V|$ learnable parameters.
- Transformer components have a large number of trainable parameters in each block
 - Multi-head attention: $4d_{model}^2$, FFNN: $2d_{model} \cdot d_{ffnn} + d_{model} + d_{ffnn}$, LayerNorm: $2d_{model}$
- FFNNs dominate model size for large d_{ffnn} , though attention heads are more time consuming (m : source or target sequence length)
 - time complexities attention $\mathcal{O}(m^2 \cdot d_{model} + m \cdot d_{model}^2)$ vs FFNN $\mathcal{O}(m \cdot d_{model} \cdot d_{ffnn})$
- Searching for more efficient solutions led to a variety of innovations

Short Quiz

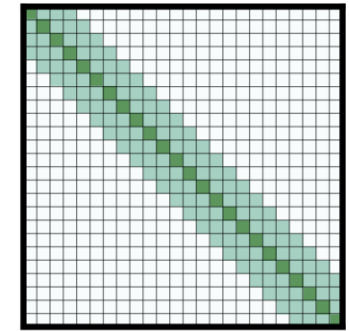
- Which of the following statements are true about **transformers**?
 1. Transformers can always be run in parallel
 2. Transformer decoders can only be parallelized during training
 3. Positional encodings help parallelize the transformer encoder
 4. Queries, keys, and values are obtained by splitting the input into 3 equal segments
 5. Multiheaded attention helps transformers find different kinds of relations between the tokens
 6. During decoding, decoder outputs function as queries and keys while the values come from the encoder

Sparse attention for handling long inputs (1)

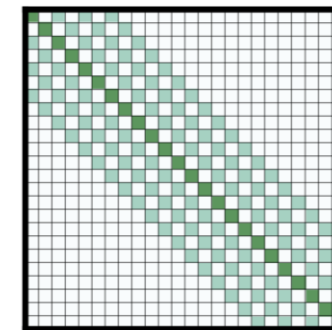
- Sliding window attention (SWA)
 - fixed window size w , each token attends to $\frac{1}{2} w$ tokens on each side $\rightarrow$ complexity $\mathcal{O}(n \cdot w)$
- Dilated sliding window attention
 - fixed window size w has gaps of size d (dilations) on each side (cf. CNN with enlarged receptive field)
- Global + SWA (cf. [Longformer](#))
 - Use SWA and add “global attention” on few pre-selected input locations symmetrically, i.e., [CLS] for classification tasks, all question tokens for QA $\rightarrow$ complexity $\mathcal{O}(n)$, but two sets of projections, Q, K, V for sliding window and global attention



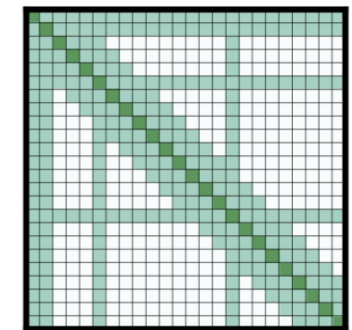
(a) Full n^2 attention



(b) Sliding window attention



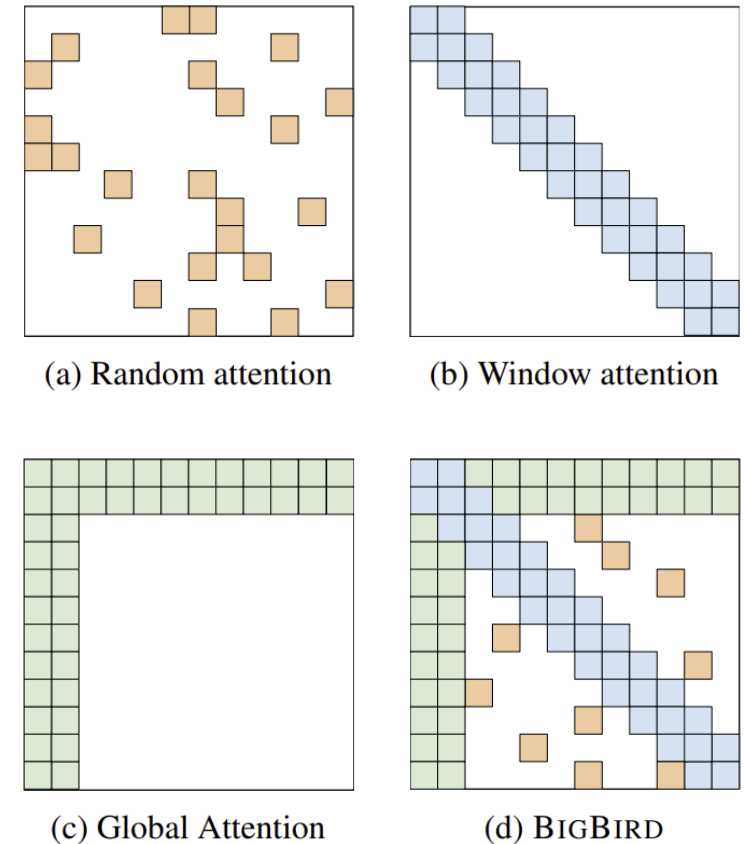
(c) Dilated sliding window



(d) Global+sliding window

Sparse attention for handling long inputs (2)

- Random + Global + SWA (cf. [Big Bird](#))
 - scales to sequence lengths of 4096 or 16,384 tokens with a small number of global (<10), random (<5) tokens and small window sizes (<< memory than standard Transformer)
 - this attention model is able to approximate any continuous function due to global tokens
 - Implementation tricks needed to translate *theoretical* into *practical* wall-clock time speedup: implement Block-Sparse attention, i.e., break the sequence into blocks to (NB: GPUs are highly optimized for dense matrix multiplication)
- Sparse attention is still relevant in classification... but not implemented in LLMs



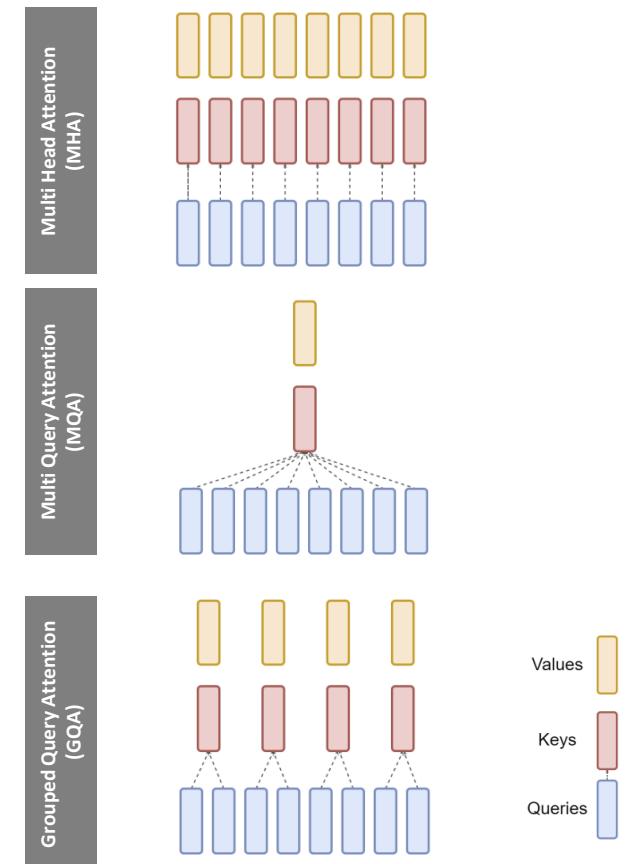
Solutions to the decoder inference bottleneck (1)

- Tokens are generated sequentially and **each new token attends to its entire history** to maintain context
- Redundant $\mathcal{O}(n^2)$ re-computation of the past, can be avoided by **caching previous keys and values** and reuse them at each decoding step (KV cache)
- KV cache saves computations, but creates a **massive memory footprint** scaling as *batch size $\times$ sequence length $\times$ hidden size $\times$ # heads*
- "**Memory Traffic**" required to load this cache from VRAM into the GPU cores becomes the primary speed limit (**memory bandwidth bottleneck**)

Step	Input Tokens	Computed K/V	Cached K/V
1	"Time"	"Time"	-
2	"Time flies"	"flies"	"Time"
3	"Time flies fast"	"fast"	"Time", "flies"

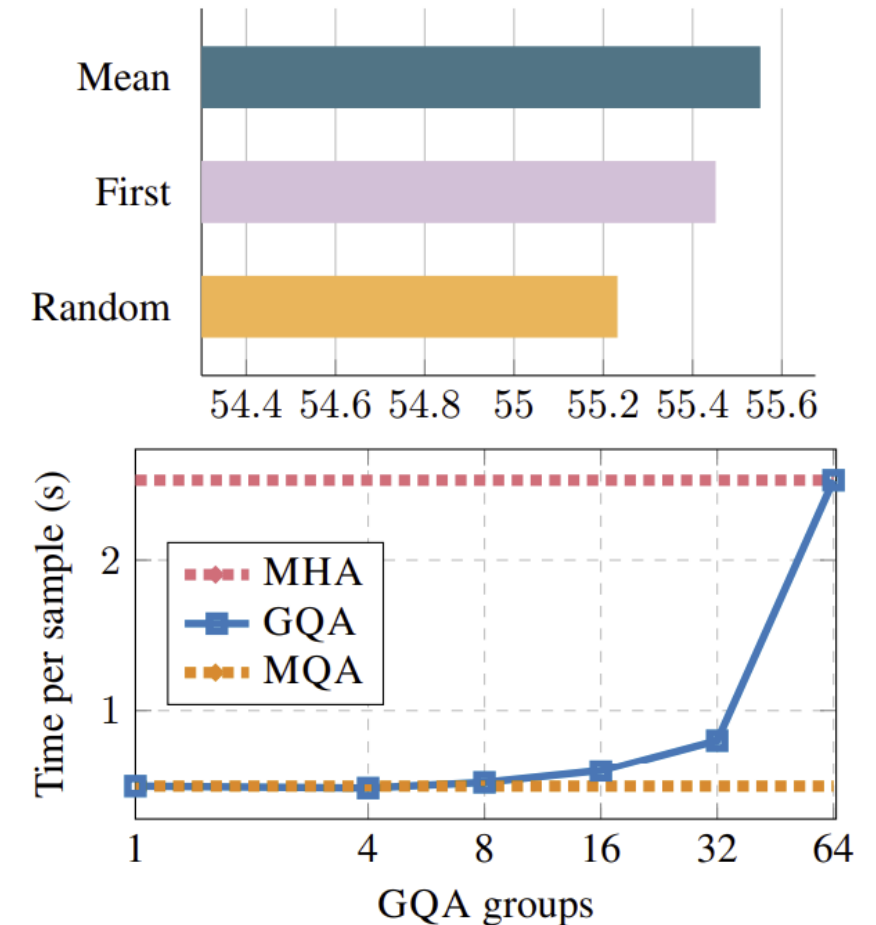
Solutions to the decoder inference bottleneck (2)

- Approaches to improve Multi head attention (MHA)
 - Multi query attention (MQA)**: all query heads share the same key and value head
 - Grouped query attention (GQA)**: query heads are divided into G groups, each of which shares a single key head and value head
 - Multi-head Latent Attention (MLA)**: query heads attend to keys and values compressed into a low-rank latent vector via matrix decomposition; this allows for on-the-fly reconstruction of full representations while drastically reducing the KV cache footprint



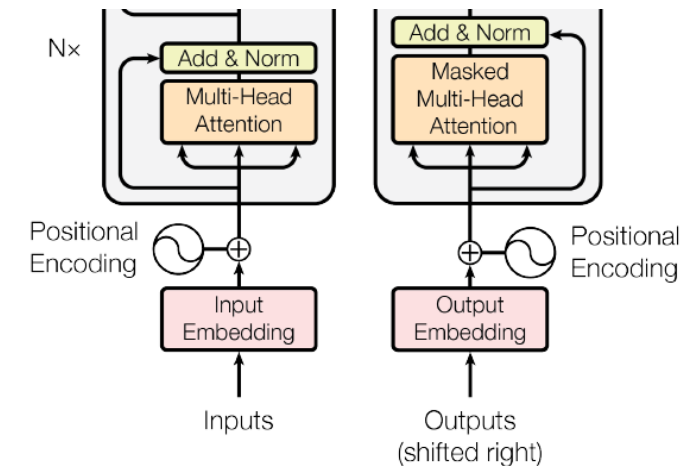
Solutions to the decoder inference bottleneck (3)

- In GQA, for each group, the original key and value projection matrices from the corresponding subset of heads are mean-pool to form one shared K and shared V for that group
- GQA results in an interpolated model that is higher quality than MQA and faster than MHA



Improvement of PE with RoPE (1)

- Trigonometric positional encodings are based on **absolute positions**, while contextual understanding depends rather on **relative token distances**
- Hence, the model must **learn** to convert “absolute indices” into “relative distances” **via attention** which is not guaranteed and is data/length dependent
- Learning absolute embeddings is tied to a maximum sequence length; trigonometric PE can extrapolate, but **performance beyond training length** can still be **unstable** because the network was optimized on a bounded range



Improvement of PE with RoPE (2)

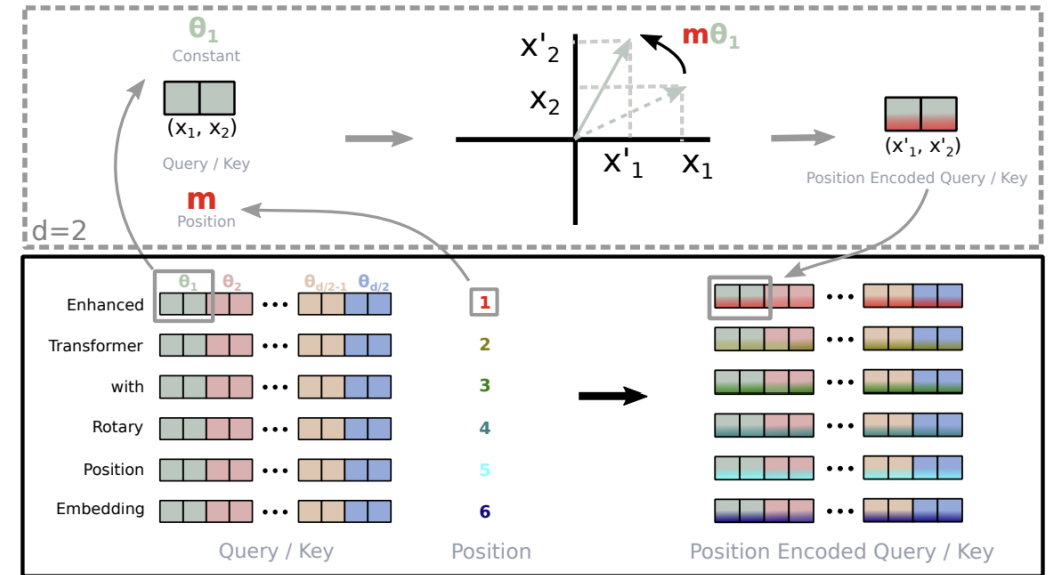
- Rotary Positional Encoding (RoPE) turns absolute positions into relative phases
 - make the query-key inner product to depend on the relative position $m - n$:

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m, -n)$$

- A solution is $f_{\{q,k\}}(x_m, m) = R_{\Theta, m}^d W_{\{q,k\}} x_m$

$$R_{\Theta, m}^d = \begin{pmatrix} block_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & block_{d_k/2} \end{pmatrix};$$

$$block_i = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}$$



Improvement of PE with RoPE (3)

- Rotary Positional Encoding (RoPE) is mainly applied in **self-attention**; for cross-attention, relative position between source and target indices is less well-defined and implementations vary
- RoPE has become **the standard positional mechanism** for state-of-the-art LLMs
 - Llama models
 - Mistral models
 - DeepSeek models

Conclusion

Transformers revolutionized and still defines modern AI

- Universal architecture for NLP representation learning via Encoder-only (see BERT)
- Foundation of modern LLMs via Decoder-only architectures (see GPT/LLaMA-class)
- Unified modeling approach across different data modalities, e.g.
 - Vision Transformers (ViT) power modern Computer Vision apps treating patches as tokens
 - Whisper drives modern Speech Recognition leveraging CNN and transformer encoders
- Backbone of Agentic AI using tools or orchestrating multiple agents